SHETH L.U.J. AND SIR M.V. COLLEGE
SUBJECT: CYBER SECURITY
PRACTICAL NO. 1

AIM: Implementing Substitution and Transposition Ciphers
1A Design and implement algorithms to encrypt and decrypt messages using classical substitution techniques.
1B Design and implement algorithms to encrypt and decrypt messages using transposition techniques.

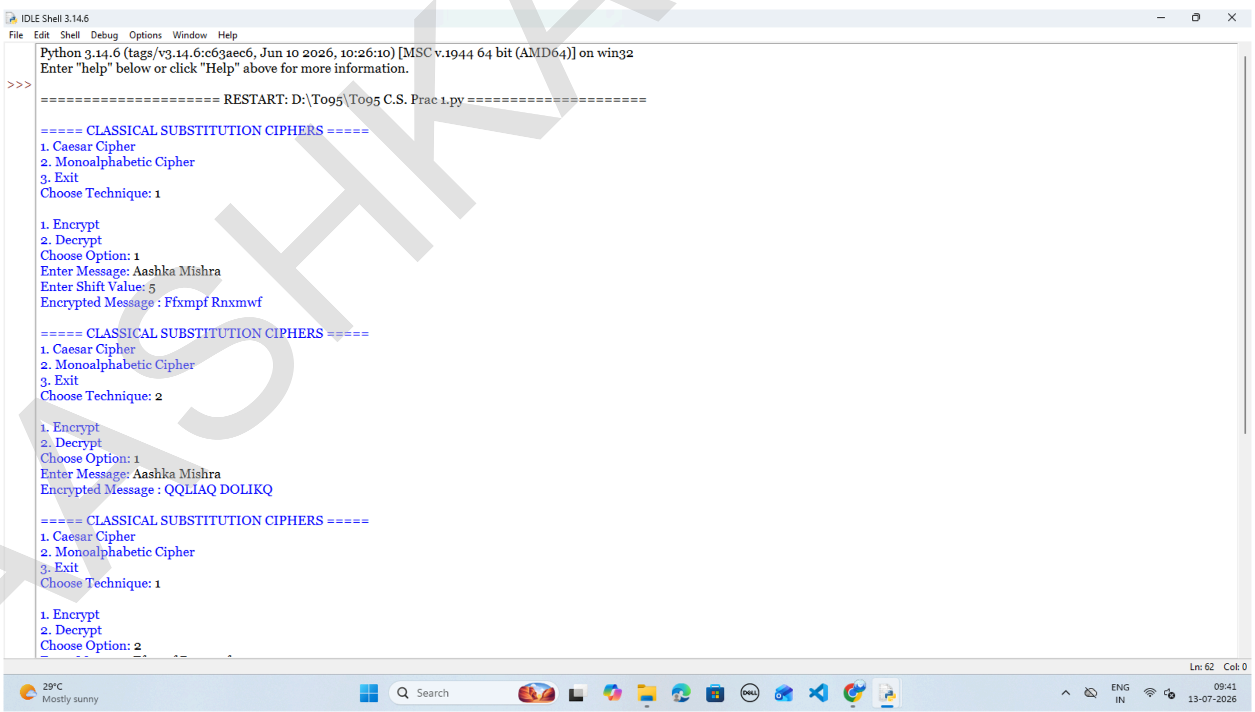
1A Design and implement algorithms to encrypt and decrypt messages using classical substitution techniques.
CODE:

```
plain = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
cipher = "QWERTYUIOPASDFGHJKLZXCVBNM"


#Caesar Cipher
def caesar_encrypt(text, shift):
    result = ""
    for ch in text:
        if ch.isalpha():
            base = ord('A') if ch.isupper() else ord('a')
            result += chr((ord(ch) - base + shift) % 26 + base)
        else:
            result += ch
    return result


def caesar_decrypt(text, shift):
    return caesar_encrypt(text, -shift)


#Monoalphabetic Cipher
def mono_encrypt(text):
    result = ""
    for ch in text.upper():
        if ch.isalpha():
            index = plain.index(ch)
            result += cipher[index]
        else:
            result += ch
    return result
```

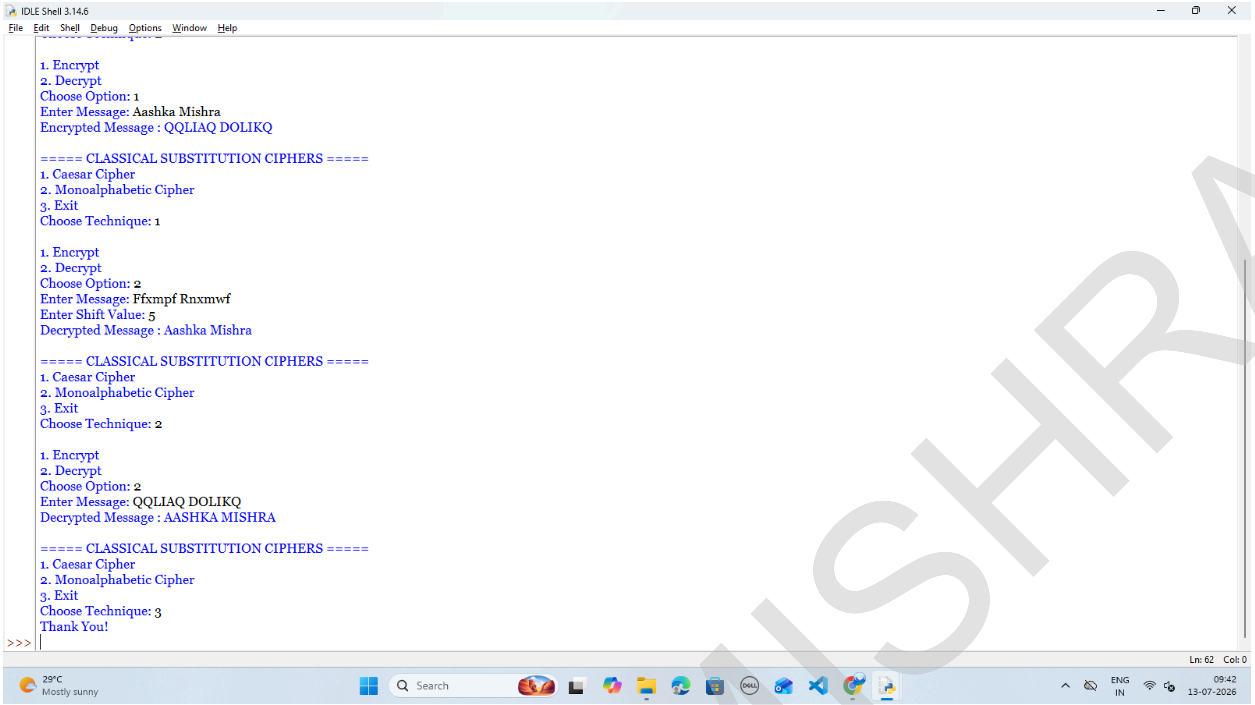
```
def mono_decrypt(text):
    result = ""
    for ch in text.upper():
        if ch.isalpha():
            index = cipher.index(ch)
            result += plain[index]
        else:
            result += ch
    return result


#Main Program
while True:

    print("\n===== CLASSICAL SUBSTITUTION CIPHERS =====")
    print("1. Caesar Cipher")
    print("2. Monoalphabetic Cipher")
    print("3. Exit")

    choice = input("Choose Technique: ")

    if choice == "1":

        print("\n1. Encrypt")
        print("2. Decrypt")

        op = input("Choose Option: ")
        msg = input("Enter Message: ")
        shift = int(input("Enter Shift Value: "))

        if op == "1":
            print("Encrypted Message :", caesar_encrypt(msg, shift))
        elif op == "2":
            print("Decrypted Message :", caesar_decrypt(msg, shift))
        else:
            print("Invalid Option!")

    elif choice == "2":
```

```
        print("\n1. Encrypt")
        print("2. Decrypt")

        op = input("Choose Option: ")
        msg = input("Enter Message: ")

        if op == "1":
            print("Encrypted Message :", mono_encrypt(msg))
        elif op == "2":
            print("Decrypted Message :", mono_decrypt(msg))
        else:
            print("Invalid Option!")

    elif choice == "3":
        print("Thank You!")
        break

    else:
        print("Invalid Choice!")
```

OUTPUT:



```
IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===================== RESTART: D:\T095\T095 C.S. Prac 1.py =====================

===== CLASSICAL SUBSTITUTION CIPHERS =====
1. Caesar Cipher
2. Monoalphabetic Cipher
3. Exit
Choose Technique: 1

1. Encrypt
2. Decrypt
Choose Option: 1
Enter Message: Aashka Mishra
Enter Shift Value: 5
Encrypted Message : Ffxmpf Rnxmwf

===== CLASSICAL SUBSTITUTION CIPHERS =====
1. Caesar Cipher
2. Monoalphabetic Cipher
3. Exit
Choose Technique: 2

1. Encrypt
2. Decrypt
Choose Option: 1
Enter Message: Aashka Mishra
Encrypted Message : QQLIAQ DOLIKQ

===== CLASSICAL SUBSTITUTION CIPHERS =====
1. Caesar Cipher
2. Monoalphabetic Cipher
3. Exit
Choose Technique: 1

1. Encrypt
2. Decrypt
Choose Option: 2
```



GUI:

```
import tkinter as tk
from tkinter import ttk, messagebox

#Monoalphabetic Key
plain = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
cipher = "QWERTYUIOPASDFGHJKLZXCVBNM"

#Caesar Cipher
def caesar_encrypt(text, shift):
    result = ""
    for ch in text:
        if ch.isalpha():
            base = ord('A') if ch.isupper() else ord('a')
            result += chr((ord(ch) - base + shift) % 26 + base)
        else:
            result += ch
    return result

def caesar_decrypt(text, shift):
    return caesar_encrypt(text, -shift)
```

```
#Monoalphabetic Cipher
def mono_encrypt(text):
    result = ""
    for ch in text.upper():
        if ch.isalpha():
            result += cipher[plain.index(ch)]
        else:
            result += ch
    return result

def mono_decrypt(text):
    result = ""
    for ch in text.upper():
        if ch.isalpha():
            result += plain[cipher.index(ch)]
        else:
            result += ch
    return result

#Functions
def encrypt():
    text = message_entry.get()

    if text == "":
        messagebox.showwarning("Warning", "Please enter a message.")
        return

    if combo.get() == "Caesar Cipher":
        try:
            shift = int(shift_entry.get())
            result.set(caesar_encrypt(text, shift))
        except:
            messagebox.showerror("Error", "Enter a valid shift value.")
    else:
        result.set(mono_encrypt(text))


def decrypt():
    text = message_entry.get()
```

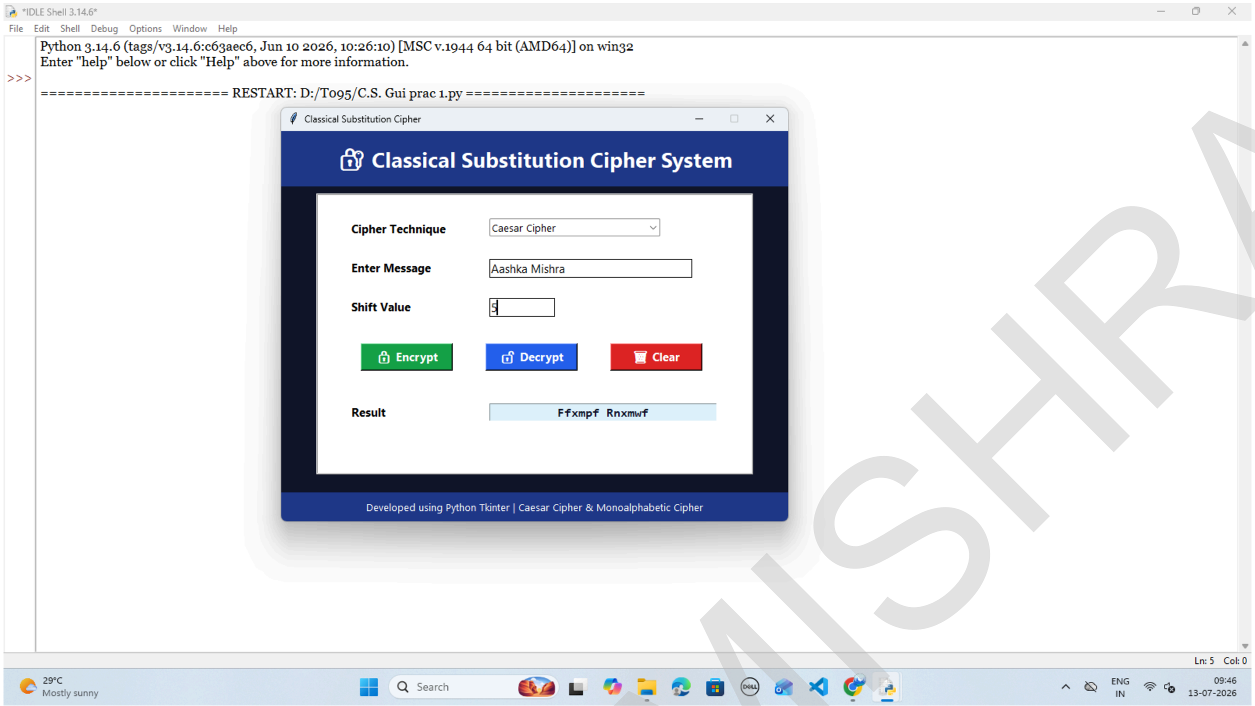
```
    if text == "":
        messagebox.showwarning("Warning", "Please enter a message.")
        return

    if combo.get() == "Caesar Cipher":
        try:
            shift = int(shift_entry.get())
            result.set(caesar_decrypt(text, shift))
        except:
            messagebox.showerror("Error", "Enter a valid shift value.")
    else:
        result.set(mono_decrypt(text))


def clear():
    message_entry.delete(0, tk.END)
    shift_entry.delete(0, tk.END)
    result.set("")


#Main Window
root = tk.Tk()
root.title("Classical Substitution Cipher")
root.geometry("650x500")
root.configure(bg="#0F172A")
root.resizable(False, False)

#Heading
title = tk.Label(
    root,
    text="🔐 Classical Substitution Cipher System",
    font=("Segoe UI", 20, "bold"),
    bg="#1E3A8A",
    fg="white",
    pady=15
)
title.pack(fill="x")

#Card
card = tk.Frame(root, bg="white", bd=2, relief="ridge")
```

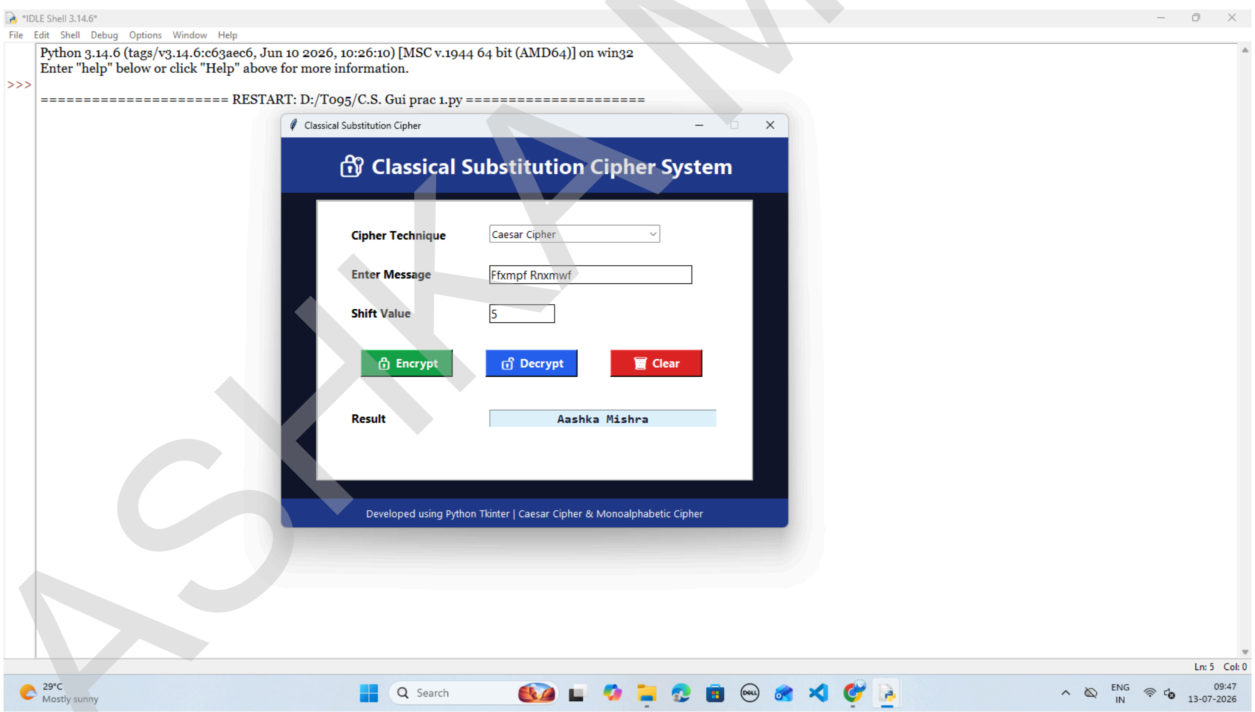
```
card.place(relx=0.5, rely=0.52, anchor="center", width=560, height=360)

#Labels
tk.Label(card,
        text="Cipher Technique",
        font=("Segoe UI", 11, "bold"),
        bg="white").place(x=40, y=30)

combo = ttk.Combobox(
    card,
    values=["Caesar Cipher", "Monoalphabetic Cipher"],
    state="readonly",
    width=28,
    font=("Segoe UI", 10)
)
combo.current(0)
combo.place(x=220, y=30)

tk.Label(card,
        text="Enter Message",
        font=("Segoe UI", 11, "bold"),
        bg="white").place(x=40, y=80)

message_entry = tk.Entry(
    card,
    font=("Segoe UI", 11),
    width=32,
    relief="solid",
    bd=1
)
message_entry.place(x=220, y=82)

tk.Label(card,
        text="Shift Value",
        font=("Segoe UI", 11, "bold"),
        bg="white").place(x=40, y=130)

shift_entry = tk.Entry(
    card,
    font=("Segoe UI", 11),
```

```
    width=10,
    relief="solid",
    bd=1
)
shift_entry.place(x=220, y=132)

#Buttons
encrypt_btn = tk.Button(
    card,
    text="🔒 Encrypt",
    font=("Segoe UI", 11, "bold"),
    bg="#16A34A",
    fg="white",
    width=12,
    command=encrypt,
    cursor="hand2"
)
encrypt_btn.place(x=55, y=190)

decrypt_btn = tk.Button(
    card,
    text="🔓 Decrypt",
    font=("Segoe UI", 11, "bold"),
    bg="#2563EB",
    fg="white",
    width=12,
    command=decrypt,
    cursor="hand2"
)
decrypt_btn.place(x=215, y=190)

clear_btn = tk.Button(
    card,
    text="🗑️ Clear",
    font=("Segoe UI", 11, "bold"),
    bg="#DC2626",
    fg="white",
    width=12,
    command=clear,
    cursor="hand2"
```

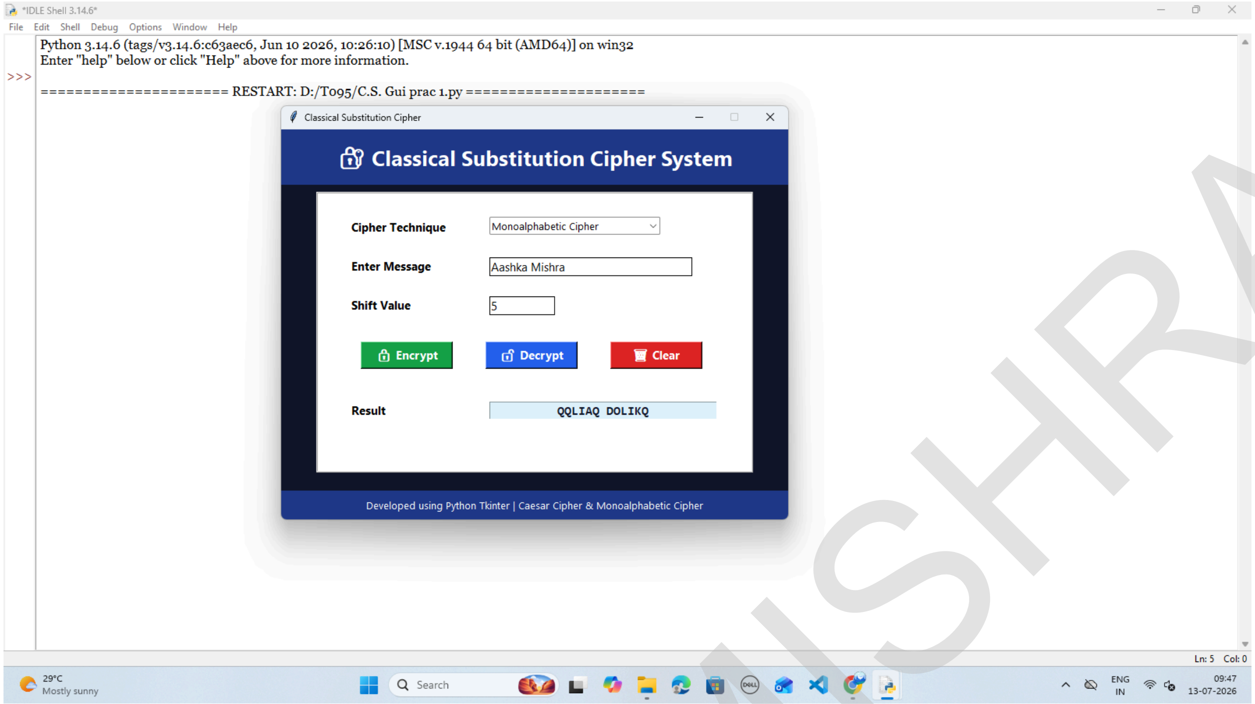
```
)
clear_btn.place(x=375, y=190)

#Result
tk.Label(card,
        text="Result",
        font=("Segoe UI", 11, "bold"),
        bg="white").place(x=40, y=265)

result = tk.StringVar()

result_box = tk.Entry(
    card,
    textvariable=result,
    font=("Consolas", 12, "bold"),
    width=32,
    justify="center",
    state="readonly",
    readonlybackground="#E0F2FE",
    fg="#0F172A"
)
result_box.place(x=220, y=267)

#Footer
footer = tk.Label(
    root,
    text="Developed using Python Tkinter | Caesar Cipher & Monoalphabetic Cipher",
    font=("Segoe UI", 10),
    bg="#1E3A8A",
    fg="white",
    pady=8
)
footer.pack(side="bottom", fill="x")

root.mainloop()
```

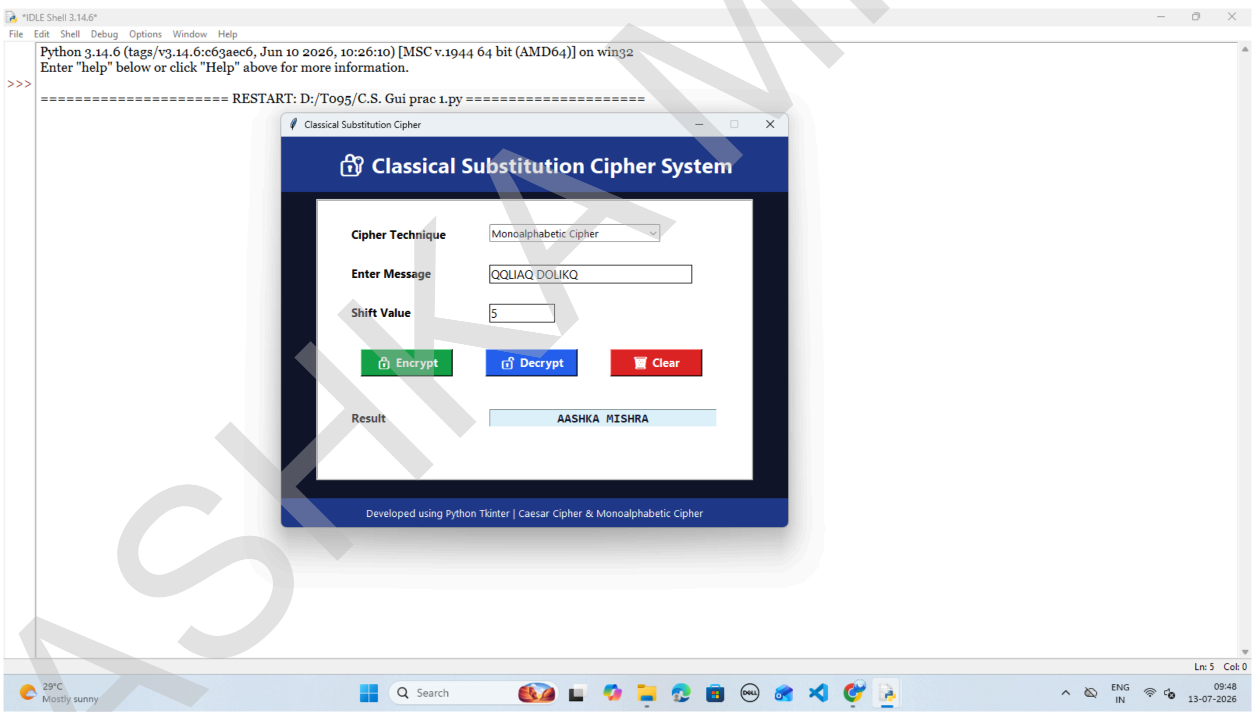
OUTPUT:


Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
==================== RESTART: D:/T095/C.S. Gui prac 1.py ====================
Classical Substitution Cipher System
Cipher Technique
Caesar Cipher
Enter Message
Aashka Mishra
Shift Value
5
Encrypt
Decrypt
Clear
Result
Ffxmpf Rnxmwf
Developed using Python Tkinter | Caesar Cipher & Monoalphabetic Cipher


Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
==================== RESTART: D:/T095/C.S. Gui prac 1.py ====================
Classical Substitution Cipher System
Cipher Technique
Caesar Cipher
Enter Message
Ffxmpf Rnxmwf
Shift Value
5
Encrypt
Decrypt
Clear
Result
Aashka Mishra
Developed using Python Tkinter | Caesar Cipher & Monoalphabetic Cipher

SHETH L.U.J. AND SIR M.V. COLLEGE
SUBJECT: CYBER SECURITY





NAME: AASHKA MISHRA
ROLL NO: T095

1B Design and implement algorithms to encrypt and decrypt messages using transposition techniques.

CODE:

```
#Rail Fence Cipher
def rail_encrypt(text, key):
    rail = [''] * key
    row, step = 0, 1

    for ch in text:
        rail[row] += ch
        if row == 0:
            step = 1
        elif row == key - 1:
            step = -1
        row += step

    return ''.join(rail)


def rail_decrypt(cipher, key):
    n = len(cipher)
    mark = [[0] * n for _ in range(key)]

    row, step = 0, 1
    for i in range(n):
        mark[row][i] = 1
        if row == 0:
            step = 1
        elif row == key - 1:
            step = -1
        row += step

    k = 0
    for i in range(key):
        for j in range(n):
            if mark[i][j]:
                mark[i][j] = cipher[k]
                k += 1

    row, step = 0, 1
```

```
    text = ""
    for i in range(n):
        text += mark[row][i]
        if row == 0:
            step = 1
        elif row == key - 1:
            step = -1
        row += step

    return text
```

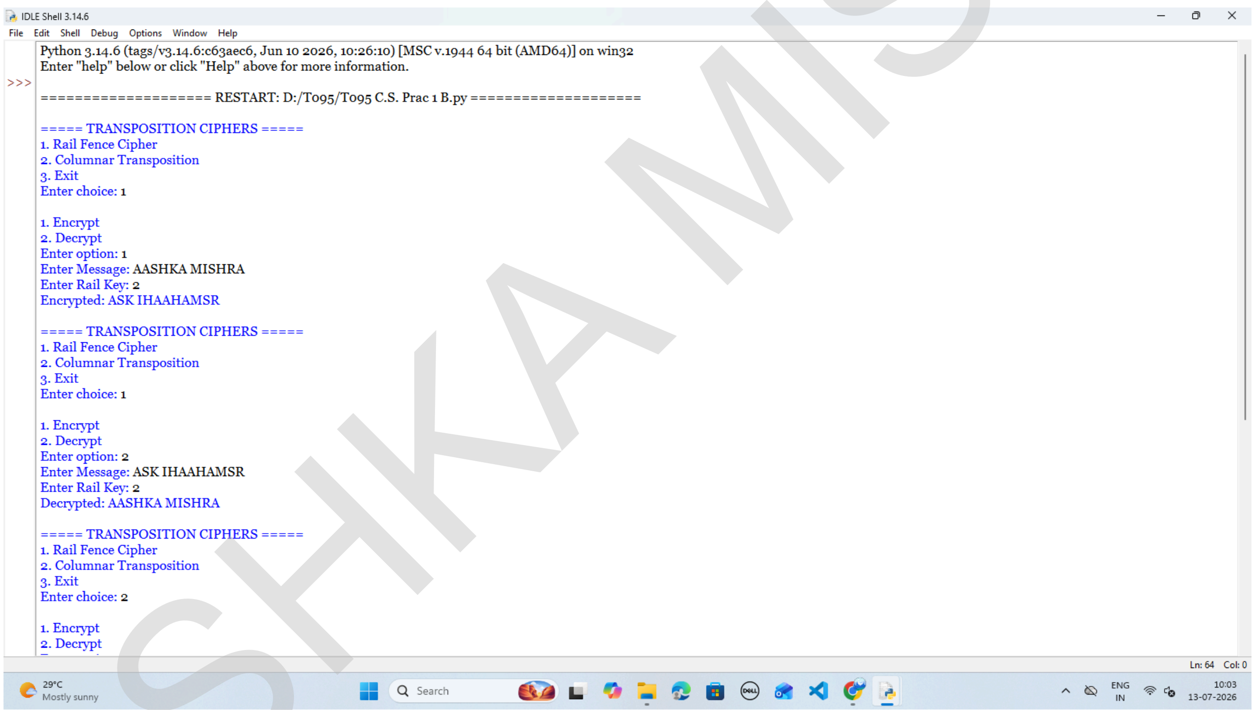
#Columnar Transposition

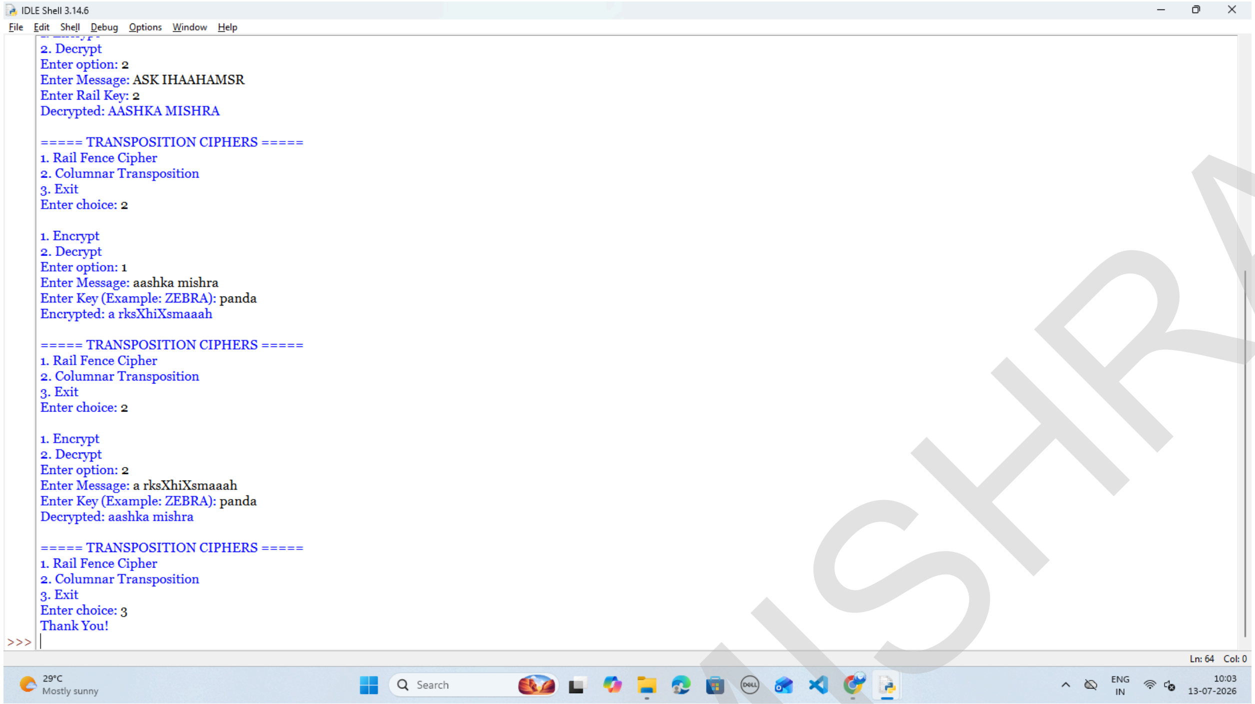
```
def col_encrypt(text, key):
    cols = len(key)
    rows = (len(text) + cols - 1) // cols

    text += "X" * (rows * cols - len(text))

    matrix = [list(text[i:i + cols]) for i in range(0, len(text), cols)]
    order = sorted(range(cols), key=lambda i: key[i])

    cipher = ""
    for c in order:
        for r in matrix:
            cipher += r[c]
    return cipher


def col_decrypt(cipher, key):
    cols = len(key)
    rows = len(cipher) // cols

    order = sorted(range(cols), key=lambda i: key[i])

    matrix = [[''] * cols for _ in range(rows)]

    k = 0
    for c in order:
        for r in range(rows):
```

```
            matrix[r][c] = cipher[k]
            k += 1

    text = ""
    for r in matrix:
        text += "".join(r)

    return text.rstrip("X")


#Main Menu
while True:

    print("\n===== TRANSPOSITION CIPHERS =====")
    print("1. Rail Fence Cipher")
    print("2. Columnar Transposition")
    print("3. Exit")

    choice = input("Enter choice: ")

    if choice == "3":
        print("Thank You!")
        break

    print("\n1. Encrypt")
    print("2. Decrypt")
    op = input("Enter option: ")

    msg = input("Enter Message: ")

    if choice == "1":
        key = int(input("Enter Rail Key: "))

        if op == "1":
            print("Encrypted:", rail_encrypt(msg, key))
        elif op == "2":
            print("Decrypted:", rail_decrypt(msg, key))
        else:
            print("Invalid Option!")
```

```
elif choice == "2":
    key = input("Enter Key (Example: ZEBRA): ").upper()

    if op == "1":
        print("Encrypted:", col_encrypt(msg, key))
    elif op == "2":
        print("Decrypted:", col_decrypt(msg, key))
    else:
        print("Invalid Option!")

else:
    print("Invalid Choice!")
```

OUTPUT:





GUI:

```
import tkinter as tk
from tkinter import ttk, messagebox

#Rail Fence Cipher
def rail_encrypt(text, key):
    rail = [''] * key
    row, step = 0, 1
    for ch in text:
        rail[row] += ch
        if row == 0:
            step = 1
        elif row == key - 1:
            step = -1
        row += step
    return ''.join(rail)

def rail_decrypt(cipher, key):
    n = len(cipher)
    mark = [[0] * n for _ in range(key)]

    row, step = 0, 1
```

```
    for i in range(n):
        mark[row][i] = 1
        if row == 0:
            step = 1
        elif row == key - 1:
            step = -1
        row += step

    k = 0
    for i in range(key):
        for j in range(n):
            if mark[i][j]:
                mark[i][j] = cipher[k]
                k += 1

    row, step = 0, 1
    text = ""
    for i in range(n):
        text += mark[row][i]
        if row == 0:
            step = 1
        elif row == key - 1:
            step = -1
        row += step

    return text
```

#Columnar Cipher

```
def col_encrypt(text, key):
    cols = len(key)
    rows = (len(text) + cols - 1) // cols

    text += "X" * (rows * cols - len(text))

    matrix = [list(text[i:i+cols]) for i in range(0, len(text), cols)]
    order = sorted(range(cols), key=lambda i: key[i])

    cipher = ""
    for c in order:
        for r in matrix:
```

```
        cipher += r[c]
    return cipher

def col_decrypt(cipher, key):
    cols = len(key)
    rows = len(cipher) // cols
    order = sorted(range(cols), key=lambda i: key[i])

    matrix = [[''] * cols for _ in range(rows)]

    k = 0
    for c in order:
        for r in range(rows):
            matrix[r][c] = cipher[k]
            k += 1

    text = ""
    for r in matrix:
        text += "".join(r)

    return text.rstrip("X")

#Functions
def encrypt():
    msg = message.get()

    if msg == "":
        messagebox.showwarning("Warning", "Enter a message!")
        return

    try:
        if cipher_type.get() == "Rail Fence Cipher":
            key = int(key_entry.get())
            result.set(rail_encrypt(msg, key))
        else:
            key = key_entry.get().upper()
            result.set(col_encrypt(msg, key))
    except:
        messagebox.showerror("Error", "Invalid Key!")
```

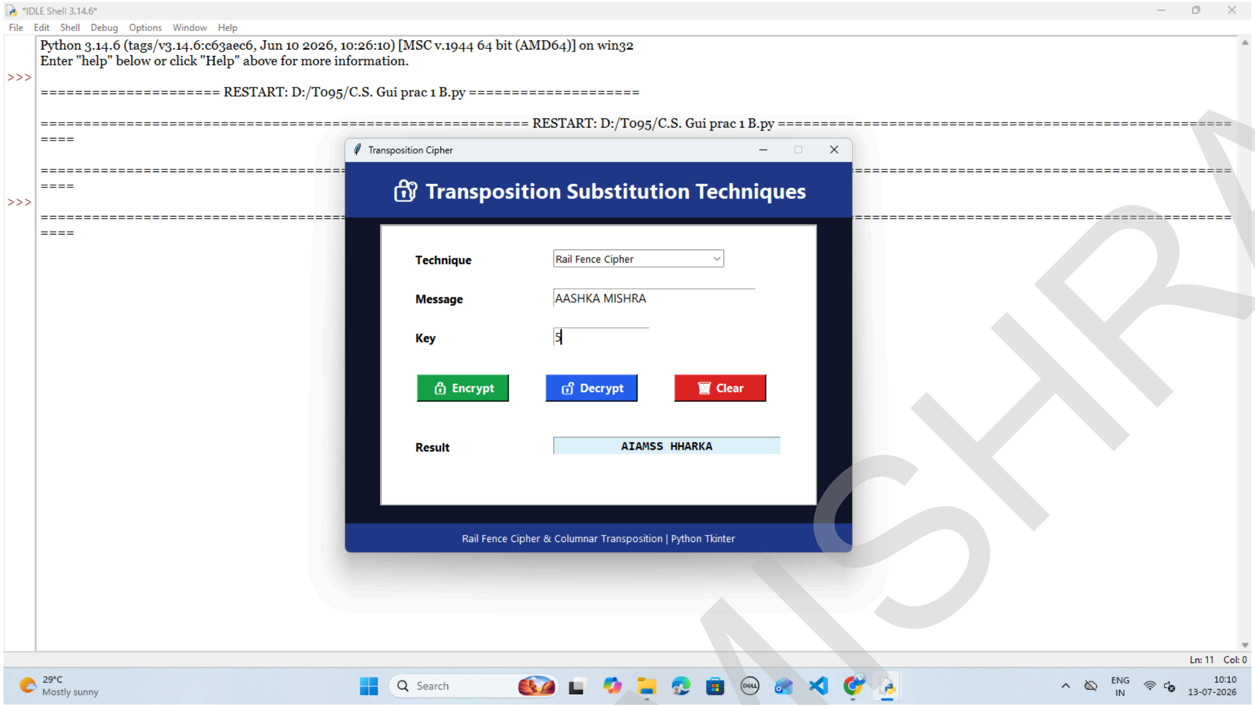
```
def decrypt():
    msg = message.get()

    if msg == "":
        messagebox.showwarning("Warning", "Enter a message!")
        return

    try:
        if cipher_type.get() == "Rail Fence Cipher":
            key = int(key_entry.get())
            result.set(rail_decrypt(msg, key))
        else:
            key = key_entry.get().upper()
            result.set(col_decrypt(msg, key))
    except:
        messagebox.showerror("Error", "Invalid Key!")

def clear():
    message.delete(0, tk.END)
    key_entry.delete(0, tk.END)
    result.set("")

#GUI
root = tk.Tk()
root.title("Transposition Cipher")
root.geometry("650x500")
root.configure(bg="#0F172A")
root.resizable(False, False)

# Header
tk.Label(
    root,
    text="🔐 Transposition Substitution Techniques",
    font=("Segoe UI", 20, "bold"),
    bg="#1E3A8A",
    fg="white",
    pady=15
).pack(fill="x")

# Card
```

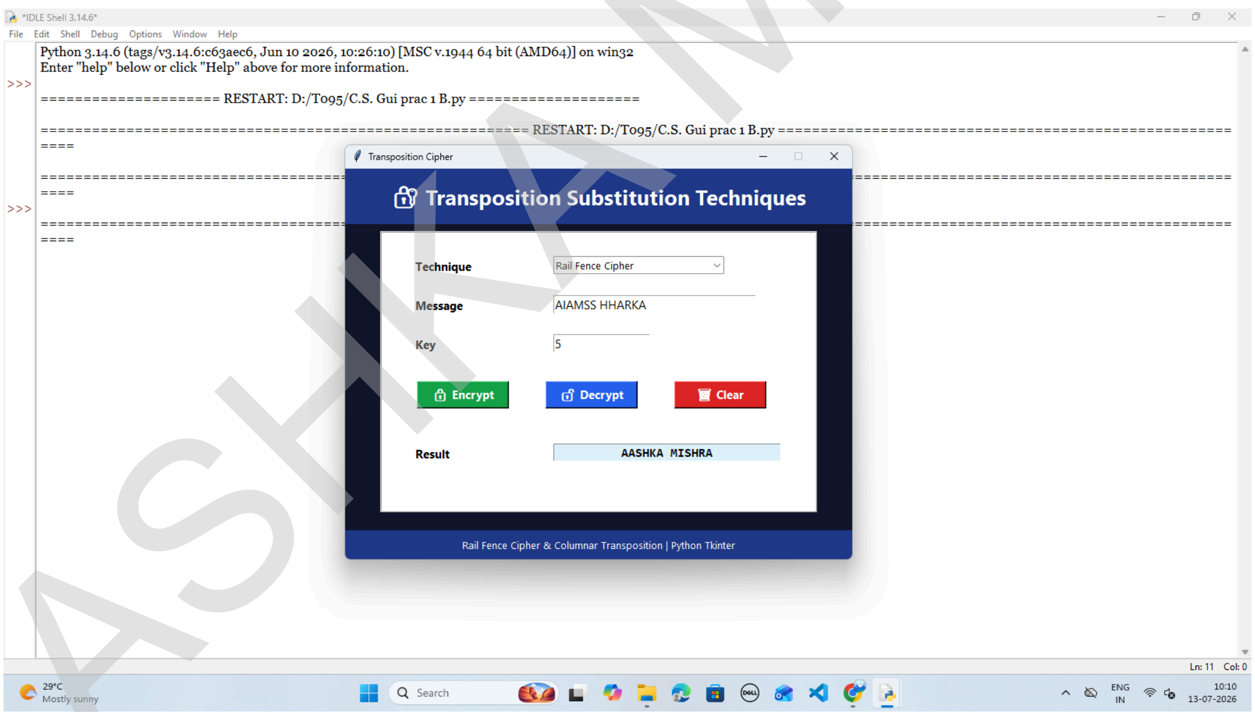
```
card = tk.Frame(root, bg="white", bd=2, relief="ridge")
card.place(relx=0.5, rely=0.52, anchor="center", width=560, height=360)

# Technique
tk.Label(card, text="Technique", font=("Segoe UI",11,"bold"),
        bg="white").place(x=40, y=30)

cipher_type = ttk.Combobox(
    card,
    values=["Rail Fence Cipher", "Columnar Transposition"],
    state="readonly",
    width=28,
    font=("Segoe UI",10)
)
cipher_type.current(0)
cipher_type.place(x=220, y=30)

# Message
tk.Label(card, text="Message", font=("Segoe UI",11,"bold"),
        bg="white").place(x=40, y=80)

message = tk.Entry(card, width=32, font=("Segoe UI",11))
message.place(x=220, y=80)

# Key
tk.Label(card, text="Key", font=("Segoe UI",11,"bold"),
        bg="white").place(x=40, y=130)

key_entry = tk.Entry(card, width=15, font=("Segoe UI",11))
key_entry.place(x=220, y=130)

# Buttons
tk.Button(card, text="🔒 Encrypt",
         bg="#16A34A", fg="white",
         font=("Segoe UI",11,"bold"),
         width=12,
         command=encrypt).place(x=45, y=190)

tk.Button(card, text="🔓 Decrypt",
         bg="#2563EB", fg="white",
```

```
          font=("Segoe UI",11,"bold"),
          width=12,
          command=decrypt).place(x=210, y=190)

tk.Button(card, text="🗑️ Clear",
          bg="#DC2626", fg="white",
          font=("Segoe UI",11,"bold"),
          width=12,
          command=clear).place(x=375, y=190)

# Result
tk.Label(card, text="Result",
         font=("Segoe UI",11,"bold"),
         bg="white").place(x=40, y=270)

result = tk.StringVar()

tk.Entry(card,
         textvariable=result,
         font=("Consolas",12,"bold"),
         width=32,
         justify="center",
         state="readonly",
         readonlybackground="#E0F2FE").place(x=220, y=270)

# Footer
tk.Label(
    root,
    text="Rail Fence Cipher & Columnar Transposition | Python Tkinter",
    bg="#1E3A8A",
    fg="white",
    font=("Segoe UI",10),
    pady=8
).pack(side="bottom", fill="x")

root.mainloop()
```

SHETH L.U.J. AND SIR M.V. COLLEGE
SUBJECT: CYBER SECURITY

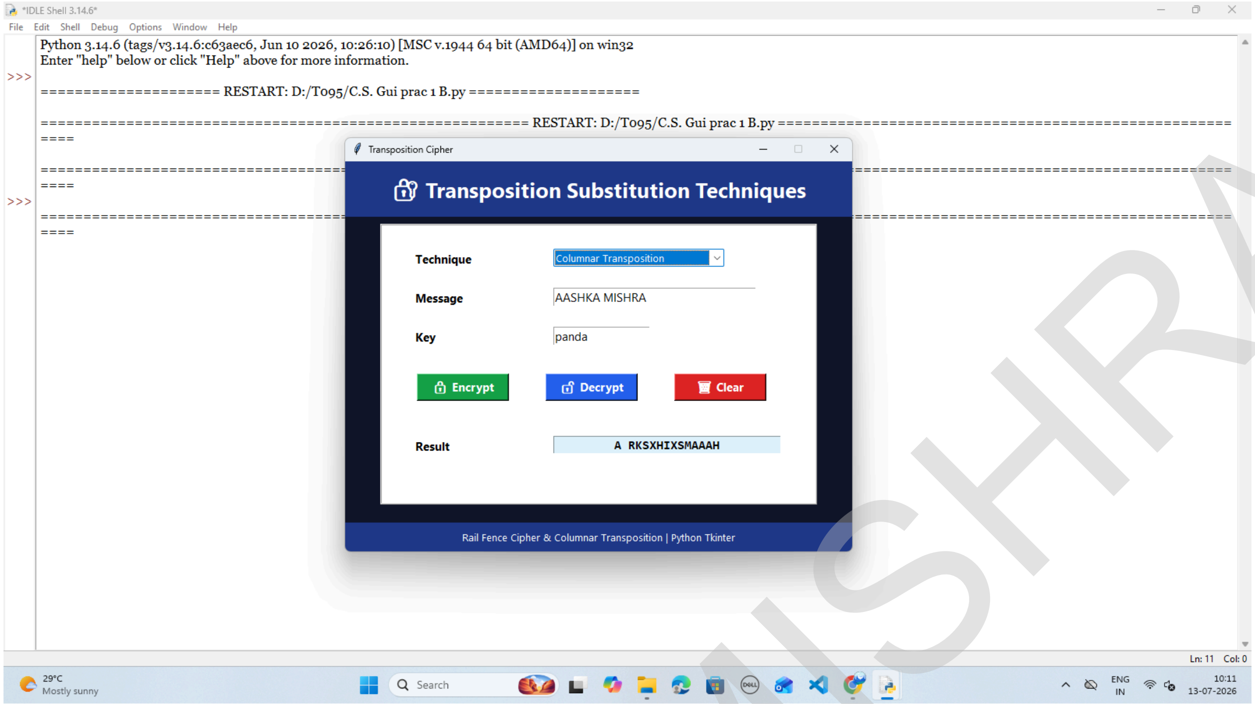
OUTPUT:

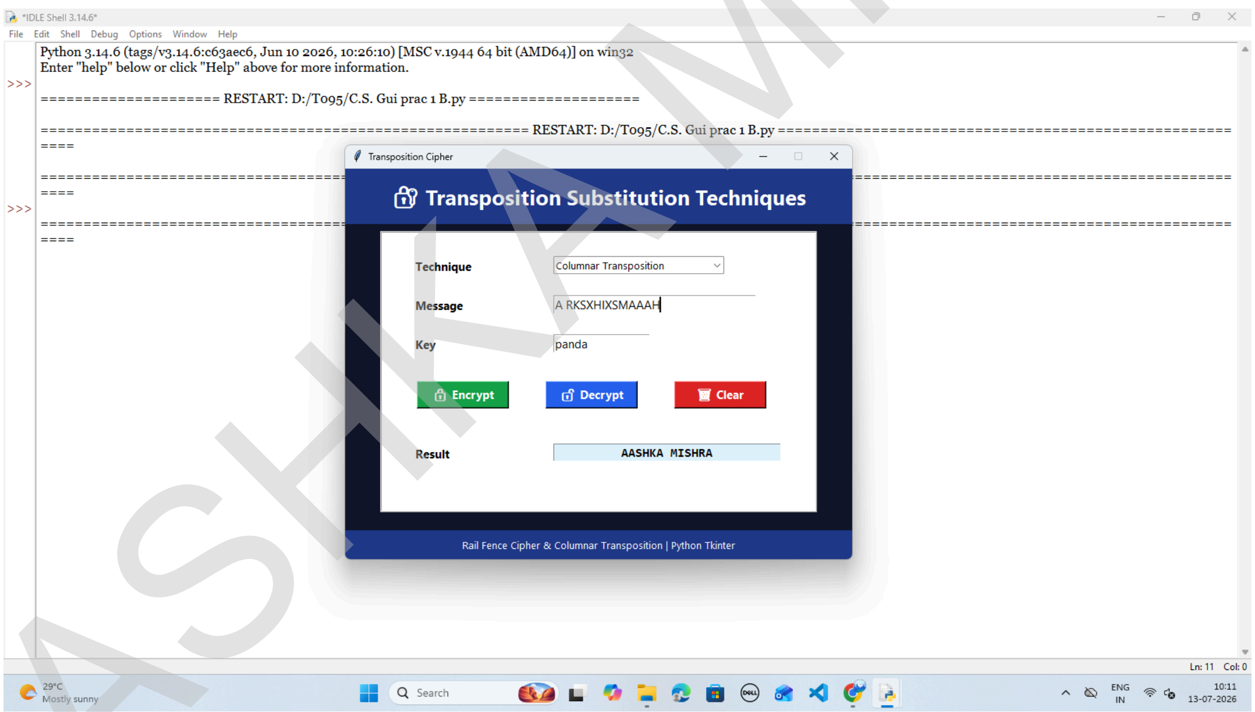




NAME: AASHKA MISHRA
ROLL NO: T095

SHETH L.U.J. AND SIR M.V. COLLEGE
SUBJECT: CYBER SECURITY





NAME: AASHKA MISHRA
ROLL NO: T095